

Day 1 — Role Baseline and Python Backend Foundations for Applied AI

Beginner-friendly summary

A Senior Applied AI/ML Engineer is not evaluated only on model knowledge. The role sits at the intersection of:

- Understanding a business or analytical problem.
- Selecting and evaluating an appropriate ML, forecasting, retrieval, or agentic solution.
- Building reliable Python services around that solution.
- Deploying, monitoring, governing, and improving it in production.
- Leading technical decisions and communicating trade-offs.
- Writing correct code under interview constraints.

Your strongest starting evidence is in **Python backend engineering, production RAG, agent workflows, APIs, Kubernetes, evaluation gates, and end-to-end ownership**. Your most important evidence gaps are **forecasting, rigorous classical-ML experimentation, deeper analytical case studies, formal AI governance, and interview-speed coding**.

For today's practical exercise, we will build a finance analytics service where:

1. An API receives an account and date range.
2. A repository supplies transactions.
3. A service calculates financial features.
4. A mocked ML client produces a risk prediction.
5. Domain errors, configuration, logging, and dependencies remain separated.

Because an exact Google requisition or job description was not included here, this baseline uses the competency areas in your prompt as the role contract rather than claiming requirements from a specific current opening.

1. Senior Applied AI/ML competency model

1.1 Analytics and problem framing

At senior level, analytics is not merely creating charts or running SQL.

You should be able to:

- Convert an ambiguous business question into measurable outcomes.
- Define the target variable, population, observation window, and prediction horizon.
- Identify whether the task is descriptive, diagnostic, predictive, causal, or prescriptive.
- Detect leakage, survivorship bias, sampling bias, and misleading aggregations.
- Select useful baselines before introducing complex models.
- Explain findings to product, engineering, finance, and leadership stakeholders.

Example

Business request:

| “Identify customers likely to experience financial stress.”

A senior engineer first clarifies:

- What does “financial stress” mean?
- Is the outcome a late payment, overdraft, loan default, or cash-flow deficit?
- How early must the system predict it?
- What action will follow the prediction?
- What is the cost of a false positive versus a false negative?
- Are any features legally or ethically inappropriate?
- How will performance be monitored across customer groups and time?

The model comes after the decision problem is defined.

1.2 Classical machine learning

You need more than familiarity with scikit-learn APIs.

Senior-level competency includes:

- Feature engineering and feature selection.
- Baseline models such as logistic regression and simple trees.
- Tree ensembles: Random Forest, XGBoost, LightGBM, or equivalent concepts.
- Cross-validation and correct splitting strategies.
- Class imbalance handling.
- Threshold selection.
- Calibration.
- Explainability.
- Offline-to-online feature consistency.
- Model drift and retraining decisions.

An interviewer may give you a tabular fraud, churn, credit-risk, pricing, or ranking problem and ask you to design the full lifecycle.

1.3 Forecasting

Forecasting is different from ordinary supervised learning because ordering in time matters.

Expected concepts include:

- Trend, seasonality, cycles, and residual noise.
- Naive and seasonal-naive baselines.
- Time-based train/validation/test splits.
- Rolling-window backtesting.

- Lag and rolling features.
- ARIMA-style models at a conceptual level.
- Tree-based forecasting.
- Deep forecasting when justified.
- Point versus probabilistic forecasts.
- Metrics such as MAE, RMSE, WAPE, MASE, and quantile loss.
- Prediction intervals.
- Hierarchical and grouped forecasting.
- Concept drift, holidays, promotions, and external regressors.

The most common senior-level mistake is using random cross-validation on time-series data.

1.4 RAG and agents

This includes more than calling an LLM.

You should understand:

- Retrieval quality before generation quality.
- Chunking, metadata, embeddings, hybrid search, reranking, and filtering.
- Tool contracts and deterministic tool execution.
- Agent state, routing, planning, retries, and termination.
- Idempotency and side-effect safety.
- Human approval for high-consequence actions.
- Prompt injection and indirect prompt injection.
- Evaluation at component and end-to-end levels.
- Model and tool fallback policies.
- Cost, latency, and observability.

Your DPDK BenchOps Copilot provides substantial evidence here: ingestion, hybrid retrieval, LangGraph orchestration, MCP tools, verification, citations, evaluation gates, Kubernetes delivery, and dependency safeguards are all documented in your resume.

1.5 Governance and responsible AI

Governance means proving that the AI system is controlled, traceable, and appropriate for its use case.

It may include:

- Data lineage and permitted-use controls.
- PII identification and minimization.
- Access control.
- Model, prompt, retrieval, and dataset versioning.
- Approval workflows.
- Audit trails.

- Fairness and subgroup evaluation.
- Explainability requirements.
- Model cards and system cards.
- Human oversight.
- Incident handling.
- Retention and deletion policies.
- Monitoring for harmful or disallowed behavior.

Guardrails alone are not complete governance. Guardrails are runtime controls; governance covers the full organizational and technical lifecycle.

1.6 Production engineering

A production AI system must satisfy two separate quality dimensions:

$$\text{Production success} = \text{AI quality} + \text{software reliability}$$

A model with excellent offline accuracy can still fail because of:

- Timeouts.
- Feature inconsistency.
- Schema changes.
- Dependency failures.
- Incorrect retries.
- Data freshness problems.
- Resource exhaustion.
- Poor release controls.
- Missing observability.
- Unsafe fallback behavior.

Production competency therefore includes APIs, asynchronous execution, databases, caching, queues, deployment, monitoring, testing, security, and failure recovery.

1.7 Leadership

For a senior role, leadership is not limited to being a people manager.

Evidence may include:

- Converting ambiguous requirements into an engineering plan.
- Writing HLDs and LLDs.
- Leading design reviews.
- Mentoring engineers.
- Creating reusable patterns.
- Negotiating scope.
- Challenging unsafe or technically weak proposals.

- Coordinating across product, data science, infrastructure, and governance teams.
- Managing incidents and learning from them.
- Explaining trade-offs to non-specialists.

Your resume states that you currently lead the IBM IC4V Metering Team and previously owned the DPDK Copilot from requirements through production rollout. That is meaningful evidence, but stronger interview stories will need team size, your actual decisions, disagreements, mentoring actions, and measurable outcomes where genuinely available.

1.8 Coding and problem solving

You should prepare for two kinds of coding:

Algorithmic coding

- Arrays, strings, hashing, stacks, queues, trees, graphs.
- Two pointers and sliding windows.
- Heaps and intervals.
- Binary search.
- Recursion and dynamic programming.
- Time and space complexity.
- Correctness and edge cases.

Production-oriented coding

- Type-safe interfaces.
- API and domain models.
- Error handling.
- Concurrency.
- Testing.
- Data transformations.
- Maintainable object-oriented or functional design.
- Extensible code under changing requirements.

A senior candidate should not produce an elaborate architecture for a 20-minute coding problem. The interviewer expects the right level of design for the context.

2. Mapping your resume to the competency model

Evidence-based mapping

Competency	Resume evidence	Current evidence gap
Analytics	IBM metering and billing context; earlier custom analysis and predictive-model responsibilities; data-intensive backend work	Few concrete examples showing hypothesis formation, SQL analysis, statistical inference, analytical findings, or business decisions

Classical ML	Scikit-learn pipelines, dataset preparation, model validation, ML inference services, CAPTCHA model work	No deeply documented tabular ML case with feature engineering, baseline comparison, cross-validation, calibration, interpretability, and drift
Forecasting	No explicit production forecasting project in the current resume	Major gap: forecasting formulation, backtesting, seasonality, intervals, and production monitoring need demonstrable project evidence
Agents	LangGraph and LangChain orchestration, routing, MCP tools, verification, deterministic execution	Need crisp explanations of state management, loop termination, idempotency, tool failure recovery, and when not to use agents
Governance	Allowlists, audit logging, citations, evaluation gates, guardrails, safe tool execution	Limited evidence for formal privacy review, fairness, model cards, lineage, human approval policies, retention, and governance ownership
Production engineering	Python, FastAPI, async/concurrency, Docker, Kubernetes, Helm, HPA, CI/CD, tracing, retries, timeouts, circuit breakers	Need deeper evidence around SLOs, capacity planning, data consistency, tenancy, authentication, disaster recovery, and incident ownership
Leadership	Current IBM metering-team leadership; DPDK project ownership; HLD/LLD and cross-functional work	Need STAR stories with team size, mentoring, conflict resolution, standards introduced, and influence beyond individual contribution
Coding	Long Python/backend history, concurrency, APIs, database schemas, performance work	DSA speed, proof of correctness, graph/tree fluency, and clean interview execution require deliberate practice

Your resume strongly supports production GenAI and backend engineering. It also records previous ML/DL pipelines, model-backed services, CAPTCHA training, high-throughput API work, Redis caching, load testing, and concurrency decisions. However, statements such as “predictive models” should not be expanded into specific algorithms, accuracy values, or business impacts unless you can verify them.

How to describe your profile honestly

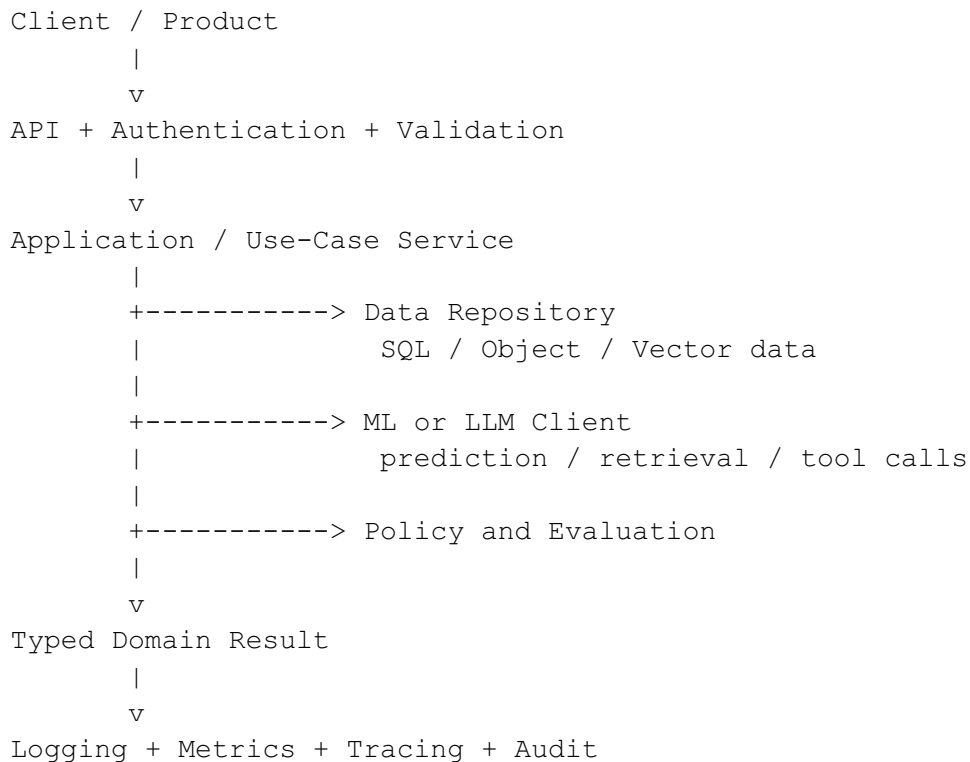
A defensible positioning statement is:

“My strongest experience is in taking AI-enabled systems from architecture through production, particularly Python services, RAG, agent orchestration, evaluation, Kubernetes deployment, and reliability. I also have experience with ML/DL pipelines and inference services, while forecasting and deeper classical-ML experimentation are areas I am deliberately strengthening through this preparation.”

That answer is confident without pretending all competencies are equally strong.

3. What backend engineering means in an applied AI platform

Backend engineering is the layer that turns a model or AI workflow into a dependable product.



Major backend responsibilities

Request handling

The service must validate:

- Input schema.
- Authentication and authorization.
- Tenant context.
- Request size.
- Date ranges.
- Supported model or operation.
- Idempotency key where side effects exist.

Orchestration

The backend decides:

- Which data to retrieve.
- Which model to invoke.
- Whether tools may be called.
- How to handle partial failures.
- When to retry.
- When to fall back.

- Whether human approval is required.

State management

AI systems often need more than stateless HTTP:

- Conversation state.
- Agent checkpoints.
- Workflow status.
- Feature snapshots.
- Model versions.
- Evaluation results.
- Audit events.

Reliability

The backend provides:

- Timeouts.
- Retries with backoff.
- Circuit breakers.
- Rate limiting.
- Caching.
- Load shedding.
- Graceful degradation.
- Health checks.
- Safe shutdown.

Observability

A production request should be traceable across:

- API receipt.
- Repository queries.
- Retrieval.
- Prompt construction.
- Model call.
- Tool calls.
- Validation.
- Final response.

Security

The service protects:

- Credentials.
- Prompts and uploaded documents.
- Financial or personal data.
- Tool permissions.

- Logs.
- Model endpoints.
- Cross-tenant boundaries.

4. How Python is used across the AI lifecycle

Stage	Typical Python work	Main engineering concern
Data preparation	Pandas, NumPy, SQL clients, validation, normalization, labeling	Reproducibility and data quality
Feature engineering	Transformations, aggregation, encoding, lag features	Leakage and training-serving consistency
Model training	scikit-learn, PyTorch, TensorFlow, XGBoost-style libraries	Experiment tracking and correct validation
Inference	Model loading, preprocessing, batching, postprocessing	Latency, memory, concurrency
RAG ingestion	Parsing, chunking, metadata, embeddings, indexing	Idempotency and document lineage
RAG query path	Retrieval, filtering, reranking, context construction	Recall, grounding, and latency
Agents	State graphs, routing, tools, retries, checkpoints	Termination, safety, and side effects
APIs	FastAPI, Pydantic, async I/O, authentication	Stable contracts and reliability
Evaluation	Metrics, golden sets, judge pipelines, regression tests	Repeatability and metric validity
Operations	CLI tools, workers, jobs, deployment automation	Configuration, monitoring, recovery

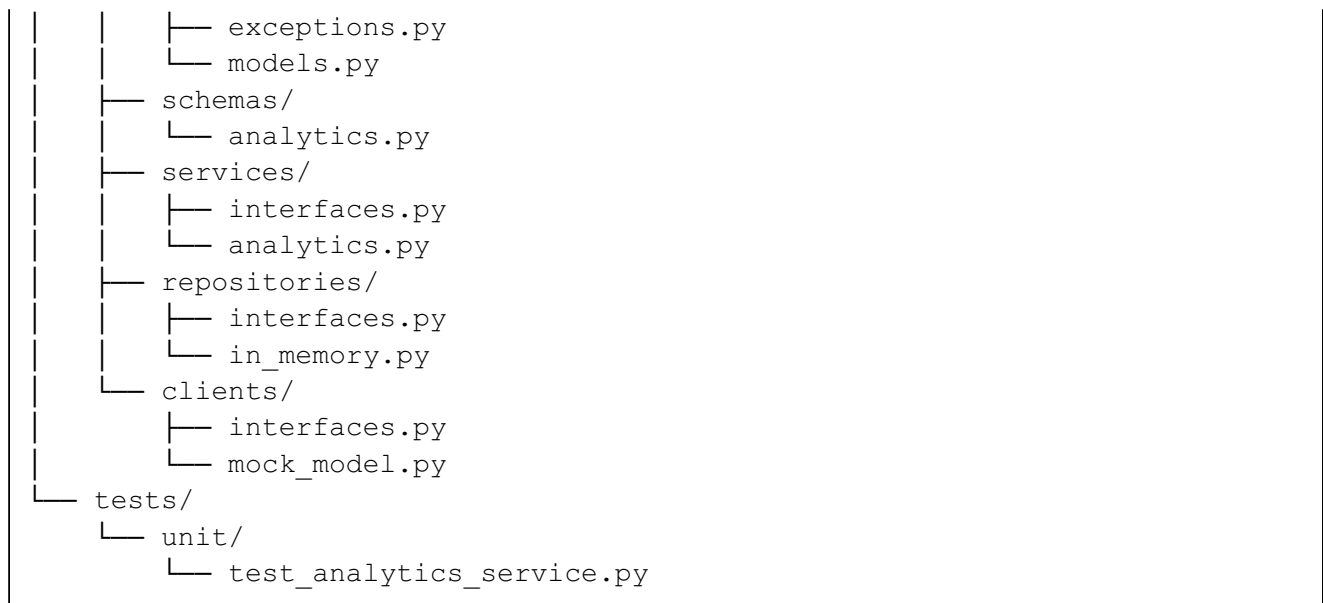
Python is effective here because it connects the data, model, orchestration, and API ecosystems. Its flexibility also creates risk: without typing, boundaries, and tests, AI code can become a collection of loosely connected notebooks and framework calls.

5. Recommended Python structure for an AI service

```

finance_ai/
├── pyproject.toml
├── app/
│   ├── main.py
│   ├── dependencies.py
│   ├── api/
│   │   └── routes.py
│   ├── config/
│   │   ├── logging.py
│   │   └── settings.py
│   └── domain/

```



Layer responsibilities

Layer	Responsibility	Must not contain
api	HTTP routing, headers, status codes, transport mapping	Core business rules
schemas	External request and response validation	Database code
domain	Business objects, invariants, domain exceptions	FastAPI or database dependencies
services	Use-case orchestration	Concrete SQL or vendor SDK details
repositories	Data-access abstractions and implementations	API response construction
clients	External model, LLM, or service calls	Business workflow ownership
config	Environment-driven configuration and logging setup	Business decisions
dependencies	Composition root that wires implementations	Domain logic
tests	Behavioral verification	Production configuration

6. Python foundations at senior interview depth

6.1 Functions

A good function should have:

- One clear responsibility.
- Explicit input and output types.
- Predictable side effects.
- A meaningful name.

- Clear failure behavior.
- Testable boundaries.

Prefer:

```
features = build_cashflow_features(transactions)
```

over:

```
process_data(transactions)
```

The first name describes the transformation and expected result.

Pure functions are especially useful for:

- Feature engineering.
- Metric calculation.
- Ranking.
- Validation.
- Postprocessing.

They are deterministic and easy to test.

6.2 Modules and packages

A **module** is normally one Python file.

```
analytics.py
```

A **package** is a directory of related modules.

```
services/  
    analytics.py  
    interfaces.py
```

Packages should represent stable responsibilities rather than arbitrary file sizes.

A weak structure:

```
utils/  
    everything.py
```

A stronger structure:

```
retrieval/  
evaluation/  
repositories/  
model_clients/
```

Avoid using `utils` as a dumping ground for unrelated logic.

6.3 Typing

Typing improves:

- IDE assistance.
- Refactoring safety.
- Interface clarity.
- Static analysis.
- Onboarding.
- Interview communication.

Example:

```
async def list_transactions(
    account_id: str,
    start: datetime,
    end: datetime,
) -> Sequence[Transaction]:
    ...
```

Important distinction:

- Type hints do not automatically enforce runtime validation.
- Pydantic validates at runtime.
- Mypy or Pyright checks many typing problems statically.

Use `Protocol` when you want structural interfaces without requiring inheritance.

6.4 Dataclasses versus Pydantic

Dataclasses

Best suited for internal domain objects:

- Lightweight.
- Standard library.
- Can be immutable.
- Keep the domain independent of API frameworks.

```
@dataclass(frozen=True, slots=True)
class RiskPrediction:
    score: float
    label: str
```

Pydantic

Best suited for system boundaries:

- API requests.
- API responses.
- Configuration.
- External payload validation.

```
class AnalysisRequest(BaseModel):  
    account_id: str  
    start: datetime  
    end: datetime
```

A useful rule is:

| Pydantic at external boundaries; domain objects inside the application.

This is not absolute, but it prevents your entire domain from becoming coupled to an API library.

6.5 Dependency injection

Dependency injection means an object receives its dependencies rather than constructing them internally.

Weak design:

```
class AnalyticsService:  
    def __init__(self):  
        self.repository = PostgresTransactionRepository()  
        self.model = VendorRiskModelClient()
```

The service is now tightly coupled to Postgres and a vendor.

Better:

```
class AnalyticsService:  
    def __init__(  
        self,  
        repository: TransactionRepository,  
        model: RiskModelClient,  
    ):  
        self.repository = repository  
        self.model = model
```

Benefits:

- Easy unit testing.
- Easy replacement of model providers.
- Clear dependencies.

- No hidden network or database creation.
- Better separation of concerns.

FastAPI's `Depends` is framework-level dependency injection. Constructor injection remains valuable inside the application.

6.6 Configuration management

Configuration should generally come from:

1. Environment variables.
2. Secret managers.
3. Deployment-specific configuration.
4. Safe defaults for local development.

Do not hardcode:

- API keys.
- Database passwords.
- Production endpoints.
- Timeouts that vary by environment.
- Feature flags.
- Model versions intended for controlled rollout.

Validate configuration at startup so that a broken environment fails fast.

7. Clean code, SOLID, and clean architecture for AI workflows

Single Responsibility Principle

Each component should have one reason to change.

- Repository changes when persistence changes.
- Model client changes when provider integration changes.
- Service changes when the business workflow changes.
- API schema changes when the external contract changes.

A service that queries SQL, builds prompts, calls an LLM, parses JSON, logs metrics, and creates HTTP responses violates this principle.

Open/Closed Principle

The orchestration should support new implementations without rewriting its core.

For example:

```
RiskModelClient
  └─ MockRiskModelClient
```

```
├─ VertexRiskModelClient
└─ LocalSklearnRiskModelClient
```

The analytics service remains unchanged if all implementations honor the same contract.

Liskov Substitution Principle

Any implementation of an interface should preserve its behavioral expectations.

If one model client returns scores between 0 and 1 while another returns percentages between 0 and 100, they are not truly substitutable even when the method signature matches.

Contracts include semantics, not just types.

Interface Segregation Principle

Prefer narrow interfaces:

```
class RiskModelClient(Protocol):
    async def predict(self, features: CashflowFeatures) -> RiskPrediction:
        ...
```

Avoid a single enormous interface containing:

- Train.
- Predict.
- Explain.
- Delete model.
- Upload dataset.
- Create endpoint.
- Search documents.
- Generate embeddings.

Callers should depend only on operations they need.

Dependency Inversion Principle

High-level workflow code depends on abstractions:

```
AnalyticsService -> TransactionRepository
AnalyticsService -> RiskModelClient
```

It does not depend directly on:

```
AnalyticsService -> psycopg
AnalyticsService -> vendor SDK
```

Clean architecture rule

Dependencies point inward:

```
Frameworks -> Adapters -> Application -> Domain
```

The domain should not know that FastAPI, Postgres, Vertex AI, or Kubernetes exists.

Practical exception

Do not create dozens of layers for a tiny service merely to demonstrate architecture. A good senior engineer balances:

- Separation.
- Complexity.
- Team size.
- Expected change.
- Operational cost.

8. Error taxonomy and safe propagation

8.1 Recommended error categories

Category	Example	Typical external result
Input validation	End date before start date	422
Authentication	Missing or invalid identity	401
Authorization/policy	User cannot access account	403
Resource/domain	Account not found	404
Conflict	Duplicate idempotency key with different payload	409
Dependency unavailable	Model endpoint unavailable	503
Dependency timeout	Model exceeded deadline	504
Rate limiting	Request quota exceeded	429
Invariant/programming error	Impossible internal state	500
Data-quality failure	Required feature missing or invalid	Depends on ownership and recoverability

8.2 Domain exceptions

Domain errors should have stable machine-readable codes:

```
class AccountNotFoundError(DomainError):
    code = "account_not_found"
    public_message = "The requested account was not found."
```

The internal message may contain operational detail, while the public message remains safe.

8.3 Safe error messages

Unsafe response:

```
{
  "error": "Postgres query failed for account 12345 using host 10.1.4.8"
}
```

Safer response:

```
{
  "code": "internal_error",
  "message": "The request could not be completed.",
  "request_id": "5de2..."
}
```

The request ID links the user-facing failure to internal logs.

8.4 Failure propagation

Use exception chaining:

```
try:
    prediction = await model.predict(features)
except TimeoutError as exc:
    raise PredictionUnavailableError("Risk model timed out") from exc
```

This provides:

- A safe domain-level exception to the caller.
- The original stack and cause for internal debugging.

Do not catch `Exception` at every layer and repeatedly log the same failure. That creates duplicate logs and obscures ownership.

A useful rule:

- Translate an exception where abstraction changes.
- Log it where the application has enough context and owns the final handling.

9. Basic structured logging for AI workflows

Traditional log:

Analysis completed

Structured log:

```
{
  "timestamp": "2026-08-06T08:30:00Z",
  "level": "INFO",
  "event": "finance_analysis_completed",
  "request_id": "abc-123",
  "transaction_count": 42,
  "model_version": "mock-risk-v1",
  "latency_ms": 31.7
}
```

Recommended AI-service fields include:

- request_id
- trace_id
- event
- tenant_id or a safe pseudonymous reference
- model_name
- model_version
- prompt_version
- retriever_version
- latency_ms
- token_count
- tool_name
- retry_count
- result_status
- error_code

Do not log raw financial records, access tokens, full prompts containing PII, or unrestricted model responses.

Logs, metrics, and traces are different

- **Logs:** detailed events.
- **Metrics:** aggregated numerical behavior, such as error rate or p95 latency.
- **Traces:** the path and duration across distributed components.

All three are important.

10. Practical task — layered finance analytics service

10.1 Problem statement

Build a service that:

1. Accepts an account ID and analysis period.
2. Retrieves transactions through a repository interface.
3. Calculates inflow, outflow, net cash flow, expense ratio, and transaction volatility.
4. Calls a mocked risk model.
5. Returns a typed result.
6. Produces structured logs.
7. Safely translates domain and dependency failures.

The mock prediction is only an architectural placeholder. It must not be described as a validated financial-risk model.

10.2 Design reasoning

The design follows these decisions:

1. **Money uses `Decimal`**, not binary floating-point arithmetic.
 2. **Domain objects are dataclasses**, keeping core logic independent of FastAPI.
 3. **Pydantic is used for transport and configuration**.
 4. **Repositories and model clients are protocols**, allowing replacement and unit testing.
 5. **The service owns orchestration**, but not HTTP or persistence details.
 6. **Timeouts are enforced around model calls**.
 7. **Domain errors contain safe public messages**.
 8. **Logging uses a request ID instead of raw account details**.
 9. **The model is explicitly named `MockRiskModelClient`** to prevent accidental claims that it is a trained or approved risk system.
-

10.3 Pseudocode

```
FUNCTION analyze(account_id, start, end, request_id):  
  
    CREATE validated analysis period  
  
    IF account does not exist:  
        RAISE AccountNotFound  
  
    transactions = repository.list_for_account(account_id, period)  
  
    IF transactions is empty:  
        RAISE NoTransactionsFound  
  
    VERIFY every transaction belongs to requested account
```

```

total_inflow = sum of credit amounts
total_outflow = sum of debit amounts
net_cash_flow = total_inflow - total_outflow

expense_ratio =
    total_outflow / total_inflow
    OR safe fallback when inflow is zero

volatility =
    population standard deviation of transaction amounts
    normalized by mean amount

features = typed feature object

TRY:
    call risk model within configured timeout
ON timeout or model failure:
    log dependency failure
    RAISE PredictionUnavailable

result = domain analysis object

log completion with:
    request ID
    transaction count
    model version
    latency

RETURN result

```

11. Code skeleton

11.1 Domain exceptions

```

# app/domain/exceptions.py

class DomainError(Exception):
    code = "domain_error"
    public_message = "The request could not be completed."

class DomainValidationError(DomainError):
    code = "invalid_domain_data"
    public_message = "The supplied data is invalid."

class InvalidPeriodError(DomainError):
    code = "invalid_period"
    public_message = "The analysis period is invalid."

class AccountNotFoundError(DomainError):

```

```

code = "account_not_found"
public_message = "The requested account was not found."

class NoTransactionsFoundError(DomainError):
    code = "no_transactions"
    public_message = "No transactions were found for the requested period."

class RepositoryUnavailableError(DomainError):
    code = "repository_unavailable"
    public_message = "Transaction data is temporarily unavailable."

class PredictionUnavailableError(DomainError):
    code = "prediction_unavailable"
    public_message = "Risk analysis is temporarily unavailable."

```

11.2 Domain models

```

# app/domain/models.py

from dataclasses import dataclass
from datetime import datetime
from decimal import Decimal
from enum import StrEnum

from app.domain.exceptions import DomainValidationError, InvalidPeriodError

class TransactionType(StrEnum):
    CREDIT = "credit"
    DEBIT = "debit"

@dataclass(frozen=True, slots=True)
class AnalysisPeriod:
    start: datetime
    end: datetime

    def __post_init__(self) -> None:
        if self.start.tzinfo is None or self.end.tzinfo is None:
            raise InvalidPeriodError("Analysis timestamps must be timezone-
aware.")

        if self.start >= self.end:
            raise InvalidPeriodError(
                "The analysis start must be earlier than the end."
            )

@dataclass(frozen=True, slots=True)

```

```

class Transaction:
    transaction_id: str
    account_id: str
    occurred_at: datetime
    amount: Decimal
    transaction_type: TransactionType

    def __post_init__(self) -> None:
        if not self.transaction_id.strip():
            raise DomainValidationError("Transaction ID cannot be empty.")

        if not self.account_id.strip():
            raise DomainValidationError("Account ID cannot be empty.")

        if self.occurred_at.tzinfo is None:
            raise DomainValidationError(
                "Transaction timestamp must be timezone-aware."
            )

        if self.amount <= Decimal("0"):
            raise DomainValidationError(
                "Transaction amount must be greater than zero."
            )

@dataclass(frozen=True, slots=True)
class CashflowFeatures:
    total_inflow: Decimal
    total_outflow: Decimal
    net_cash_flow: Decimal
    expense_ratio: float
    normalized_volatility: float
    transaction_count: int

    def to_model_payload(self) -> dict[str, float | int]:
        return {
            "total_inflow": float(self.total_inflow),
            "total_outflow": float(self.total_outflow),
            "net_cash_flow": float(self.net_cash_flow),
            "expense_ratio": self.expense_ratio,
            "normalized_volatility": self.normalized_volatility,
            "transaction_count": self.transaction_count,
        }

@dataclass(frozen=True, slots=True)
class RiskPrediction:
    score: float
    label: str
    model_version: str

    def __post_init__(self) -> None:
        if not 0.0 <= self.score <= 1.0:
            raise DomainValidationError(

```

```

        "Risk score must be between zero and one."
    )

    if not self.model_version.strip():
        raise DomainValidationError("Model version cannot be empty.")

@dataclass(frozen=True, slots=True)
class FinanceAnalysis:
    request_id: str
    account_id: str
    period: AnalysisPeriod
    features: CashflowFeatures
    prediction: RiskPrediction

```

Non-obvious choice

Transaction amounts are always positive. `TransactionType` determines whether the amount is an inflow or outflow.

This avoids mixing two representations:

- Negative amount plus `DEBIT`.
- Positive amount plus `DEBIT`.

Allowing both creates double-negation bugs.

11.3 Repository interface

```

# app/repositories/interfaces.py

from typing import Protocol, Sequence

from app.domain.models import AnalysisPeriod, Transaction

class TransactionRepository(Protocol):
    async def account_exists(self, account_id: str) -> bool:
        """Return whether the account is known to the repository."""
        ...

    async def list_for_account(
        self,
        account_id: str,
        period: AnalysisPeriod,
    ) -> Sequence[Transaction]:
        """Return transactions occurring within the requested period."""
        ...

```

11.4 Model-client interface

```
# app/clients/interfaces.py

from typing import Protocol

from app.domain.models import CashflowFeatures, RiskPrediction

class RiskModelClient(Protocol):
    async def predict(
        self,
        features: CashflowFeatures,
    ) -> RiskPrediction:
        ...
```

11.5 Service interface

```
# app/services/interfaces.py

from datetime import datetime
from typing import Protocol

from app.domain.models import FinanceAnalysis

class FinanceAnalyticsUseCase(Protocol):
    async def analyze(
        self,
        *,
        account_id: str,
        start: datetime,
        end: datetime,
        request_id: str,
    ) -> FinanceAnalysis:
        ...
```

11.6 Mock repository

```
# app/repositories/in_memory.py

from collections.abc import Sequence

from app.domain.models import AnalysisPeriod, Transaction

class InMemoryTransactionRepository:
    def __init__(self, transactions: Sequence[Transaction]) -> None:
        self._transactions = tuple(transactions)
        self._account_ids = {
```

```

        transaction.account_id for transaction in self._transactions
    }

    async def account_exists(self, account_id: str) -> bool:
        return account_id in self._account_ids

    async def list_for_account(
        self,
        account_id: str,
        period: AnalysisPeriod,
    ) -> Sequence[Transaction]:
        return tuple(
            transaction
            for transaction in self._transactions
            if transaction.account_id == account_id
            and period.start <= transaction.occurred_at < period.end
        )

```

Correctness condition

The interval is half-open: $[start, end)$.

A transaction exactly at `start` is included. A transaction exactly at `end` is excluded.

Half-open intervals prevent double counting when adjacent periods are queried:

```

January:  [Jan 1, Feb 1)
February: [Feb 1, Mar 1)

```

11.7 Mock model client

```

# app/clients/mock_model.py

import asyncio

from app.domain.models import CashflowFeatures, RiskPrediction

class MockRiskModelClient:
    """
    Deterministic architectural mock.

    This is not a trained, calibrated, approved, or production financial
    model.
    """

    MODEL_VERSION = "mock-risk-v1"

    async def predict(
        self,
        features: CashflowFeatures,
    ) -> RiskPrediction:

```

```

# Simulates an asynchronous network/model call.
await asyncio.sleep(0.01)

expense_component = min(features.expense_ratio, 2.0) / 2.0
volatility_component = min(
    features.normalized_volatility,
    2.0,
) / 2.0
negative_cashflow_component = (
    1.0 if features.net_cash_flow < 0 else 0.0
)

raw_score = (
    0.65 * expense_component
    + 0.25 * volatility_component
    + 0.10 * negative_cashflow_component
)

score = round(max(0.0, min(raw_score, 1.0)), 4)

if score >= 0.70:
    label = "high"
elif score >= 0.40:
    label = "medium"
else:
    label = "low"

return RiskPrediction(
    score=score,
    label=label,
    model_version=self.MODEL_VERSION,
)

```

The clamping operation ensures this invariant: $0 \leq \text{score} \leq 1$.

The thresholds are placeholders, not statistically validated decision boundaries.

11.8 Analytics-service implementation

```

# app/services/analytics.py

import asyncio
import logging
from datetime import datetime
from decimal import Decimal
from statistics import pstdev
from time import perf_counter

from app.clients.interfaces import RiskModelClient
from app.domain.exceptions import (
    AccountNotFoundError,
    NoTransactionsFoundError,
)

```

```

        PredictionUnavailableError,
    )
from app.domain.models import (
    AnalysisPeriod,
    CashflowFeatures,
    FinanceAnalysis,
    Transaction,
    TransactionType,
)
from app.repositories.interfaces import TransactionRepository

logger = logging.getLogger(__name__)

class AnalyticsService:
    def __init__(
        self,
        repository: TransactionRepository,
        model_client: RiskModelClient,
        model_timeout_seconds: float,
    ) -> None:
        if model_timeout_seconds <= 0:
            raise ValueError("Model timeout must be greater than zero.")

        self._repository = repository
        self._model_client = model_client
        self._model_timeout_seconds = model_timeout_seconds

    async def analyze(
        self,
        *,
        account_id: str,
        start: datetime,
        end: datetime,
        request_id: str,
    ) -> FinanceAnalysis:
        started_at = perf_counter()
        period = AnalysisPeriod(start=start, end=end)

        if not await self._repository.account_exists(account_id):
            raise AccountNotFoundError(
                f"Account {account_id!r} does not exist."
            )

        transactions = await self._repository.list_for_account(
            account_id,
            period,
        )

        if not transactions:
            raise NoTransactionsFoundError(
                "No transactions exist within the requested period."
            )

```

```

self._validate_repository_result(
    expected_account_id=account_id,
    transactions=transactions,
)

features = self._build_features(transactions)

try:
    async with asyncio.timeout(self._model_timeout_seconds):
        prediction = await self._model_client.predict(features)
except TimeoutError as exc:
    logger.warning(
        "risk_model_timeout",
        extra={
            "event": "risk_model_timeout",
            "request_id": request_id,
            "timeout_seconds": self._model_timeout_seconds,
        },
    )
    raise PredictionUnavailableError(
        "Risk-model prediction exceeded its deadline."
    ) from exc
except Exception as exc:
    logger.exception(
        "risk_model_failed",
        extra={
            "event": "risk_model_failed",
            "request_id": request_id,
            "dependency": "risk_model",
        },
    )
    raise PredictionUnavailableError(
        "Risk-model prediction failed."
    ) from exc

result = FinanceAnalysis(
    request_id=request_id,
    account_id=account_id,
    period=period,
    features=features,
    prediction=prediction,
)

latency_ms = round((perf_counter() - started_at) * 1000, 2)

logger.info(
    "finance_analysis_completed",
    extra={
        "event": "finance_analysis_completed",
        "request_id": request_id,
        "transaction_count": features.transaction_count,
        "model_version": prediction.model_version,
        "risk_label": prediction.label,
    },
)

```

```

        "latency_ms": latency_ms,
    },
)

return result

@staticmethod
def _validate_repository_result(
    *,
    expected_account_id: str,
    transactions: tuple[Transaction, ...]
    | list[Transaction]
    | object,
) -> None:
    for transaction in transactions:
        if transaction.account_id != expected_account_id:
            raise RuntimeError(
                "Repository returned a transaction belonging "
                "to another account."
            )

@staticmethod
def _build_features(
    transactions: tuple[Transaction, ...]
    | list[Transaction]
    | object,
) -> CashflowFeatures:
    transaction_list = list(transactions)

    total_inflow = sum(
        (
            transaction.amount
            for transaction in transaction_list
            if transaction.transaction_type == TransactionType.CREDIT
        ),
        Decimal("0"),
    )

    total_outflow = sum(
        (
            transaction.amount
            for transaction in transaction_list
            if transaction.transaction_type == TransactionType.DEBIT
        ),
        Decimal("0"),
    )

    net_cash_flow = total_inflow - total_outflow

    expense_ratio = (
        float(total_outflow / total_inflow)
        if total_inflow > 0
        else 1.0
    )

```

```

numeric_amounts = [
    float(transaction.amount) for transaction in transaction_list
]
average_amount = sum(numeric_amounts) / len(numeric_amounts)

normalized_volatility = (
    pstdev(numeric_amounts) / average_amount
    if len(numeric_amounts) > 1 and average_amount > 0
    else 0.0
)

return CashflowFeatures(
    total_inflow=total_inflow,
    total_outflow=total_outflow,
    net_cash_flow=net_cash_flow,
    expense_ratio=expense_ratio,
    normalized_volatility=normalized_volatility,
    transaction_count=len(transaction_list),
)

```

Important design detail

The repository result is checked for cross-account contamination. In a financial or multi-tenant platform, returning another account's data is not merely a calculation bug; it is a potential security incident.

In production, account or tenant isolation should also be enforced inside the database query and authorization layer. The service-level check is defense in depth, not a substitute for access control.

11.9 Pydantic API schemas

```

# app/schemas/analytics.py

from datetime import datetime
from decimal import Decimal

from pydantic import BaseModel, Field, field_validator

from app.domain.models import FinanceAnalysis

class AnalysisRequest(BaseModel):
    account_id: str = Field(min_length=1, max_length=128)
    start: datetime
    end: datetime

    @field_validator("account_id")
    @classmethod
    def normalize_account_id(cls, value: str) -> str:
        normalized = value.strip()

```

```

        if not normalized:
            raise ValueError("Account ID cannot be blank.")
        return normalized

class RiskResponse(BaseModel):
    score: float = Field(ge=0.0, le=1.0)
    label: str
    model_version: str

class AnalysisResponse(BaseModel):
    request_id: str
    account_id: str
    start: datetime
    end: datetime
    total_inflow: Decimal
    total_outflow: Decimal
    net_cash_flow: Decimal
    expense_ratio: float
    normalized_volatility: float
    transaction_count: int
    risk: RiskResponse

    @classmethod
    def from_domain(
        cls,
        analysis: FinanceAnalysis,
    ) -> "AnalysisResponse":
        return cls(
            request_id=analysis.request_id,
            account_id=analysis.account_id,
            start=analysis.period.start,
            end=analysis.period.end,
            total_inflow=analysis.features.total_inflow,
            total_outflow=analysis.features.total_outflow,
            net_cash_flow=analysis.features.net_cash_flow,
            expense_ratio=analysis.features.expense_ratio,
            normalized_volatility=(
                analysis.features.normalized_volatility
            ),
            transaction_count=analysis.features.transaction_count,
            risk=RiskResponse(
                score=analysis.prediction.score,
                label=analysis.prediction.label,
                model_version=analysis.prediction.model_version,
            ),
        )

```

Pydantic validates the HTTP payload, but `AnalysisPeriod` also validates its own invariant. This duplication is intentional because domain logic may later be called from:

- A worker.

- A CLI.
- A scheduled job.
- A message consumer.
- Another service.

The domain should remain correct even without FastAPI.

11.10 Configuration

```
# app/config/settings.py

from functools import lru_cache

from pydantic import Field
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env",
        env_prefix="FINANCE_",
        extra="ignore",
    )

    environment: str = "local"
    log_level: str = "INFO"
    model_timeout_seconds: float = Field(default=2.0, gt=0)

@lru_cache
def get_settings() -> Settings:
    return Settings()
```

Example environment variables:

```
FINANCE_ENVIRONMENT=production
FINANCE_LOG_LEVEL=INFO
FINANCE_MODEL_TIMEOUT_SECONDS=1.5
```

Do not cache request-specific data in `get_settings`. Application settings are appropriate because they are process-level configuration.

11.11 Structured logging

```
# app/config/logging.py

import json
import logging
from datetime import UTC, datetime
from typing import Any
```

```

_STANDARD_FIELDS = set(
    logging.LogRecord(
        name="",
        level=0,
        pathname="",
        lineno=0,
        msg="",
        args=(),
        exc_info=None,
    ).__dict__
)

class JsonFormatter(logging.Formatter):
    def format(self, record: logging.LogRecord) -> str:
        payload: dict[str, Any] = {
            "timestamp": datetime.now(UTC).isoformat(),
            "level": record.levelname,
            "logger": record.name,
            "message": record.getMessage(),
        }

        for key, value in record.__dict__.items():
            if key not in _STANDARD_FIELDS and not key.startswith("_"):
                payload[key] = value

        if record.exc_info:
            payload["exception"] = self.formatException(record.exc_info)

        return json.dumps(payload, default=str)

def configure_logging(level: str) -> None:
    handler = logging.StreamHandler()
    handler.setFormatter(JsonFormatter())

    root_logger = logging.getLogger()
    root_logger.handlers.clear()
    root_logger.addHandler(handler)
    root_logger.setLevel(level.upper())

```

In a real financial service, exception logs must be reviewed to ensure SDK or database exceptions do not accidentally expose secrets or sensitive payloads.

11.12 Dependency composition

```

# app/dependencies.py

from datetime import UTC, datetime
from decimal import Decimal

```

```

from app.clients.mock_model import MockRiskModelClient
from app.config.settings import get_settings
from app.domain.models import Transaction, TransactionType
from app.repositories.in_memory import InMemoryTransactionRepository
from app.services.analytics import AnalyticsService
from app.services.interfaces import FinanceAnalyticsUseCase

_settings = get_settings()

_repository = InMemoryTransactionRepository(
    transactions=[
        Transaction(
            transaction_id="txn-001",
            account_id="account-001",
            occurred_at=datetime(2026, 8, 1, 9, 0, tzinfo=UTC),
            amount=Decimal("50000.00"),
            transaction_type=TransactionType.CREDIT,
        ),
        Transaction(
            transaction_id="txn-002",
            account_id="account-001",
            occurred_at=datetime(2026, 8, 2, 11, 30, tzinfo=UTC),
            amount=Decimal("18000.00"),
            transaction_type=TransactionType.DEBIT,
        ),
        Transaction(
            transaction_id="txn-003",
            account_id="account-001",
            occurred_at=datetime(2026, 8, 3, 16, 15, tzinfo=UTC),
            amount=Decimal("9000.00"),
            transaction_type=TransactionType.DEBIT,
        ),
    ]
)

_model_client = MockRiskModelClient()

_analytics_service = AnalyticsService(
    repository=_repository,
    model_client=_model_client,
    model_timeout_seconds=_settings.model_timeout_seconds,
)

def get_analytics_service() -> FinanceAnalyticsUseCase:
    return _analytics_service

```

This file is the **composition root**. It is the one place that decides which concrete implementations are active.

In production, it could wire:

```
PostgresTransactionRepository
VertexRiskModelClient
RedisCache
OpenTelemetryTracer
```

The domain and service logic would not need to change.

11.13 API route

```
# app/api/routes.py

from uuid import uuid4

from fastapi import APIRouter, Depends, Request

from app.dependencies import get_analytics_service
from app.schemas.analytics import AnalysisRequest, AnalysisResponse
from app.services.interfaces import FinanceAnalyticsUseCase

router = APIRouter()

@router.post(
    "/v1/analytics",
    response_model=AnalysisResponse,
)
async def analyze_finances(
    payload: AnalysisRequest,
    request: Request,
    service: FinanceAnalyticsUseCase = Depends(
        get_analytics_service
    ),
) -> AnalysisResponse:
    request_id = (
        request.headers.get("X-Request-ID")
        or str(uuid4())
    )

    result = await service.analyze(
        account_id=payload.account_id,
        start=payload.start,
        end=payload.end,
        request_id=request_id,
    )

    return AnalysisResponse.from_domain(result)
```

11.14 Application and error translation

```
# app/main.py

import logging

from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

from app.api.routes import router
from app.config.logging import configure_logging
from app.config.settings import get_settings
from app.domain.exceptions import (
    AccountNotFoundError,
    DomainError,
    InvalidPeriodError,
    NoTransactionsFoundError,
    PredictionUnavailableError,
    RepositoryUnavailableError,
)

settings = get_settings()
configure_logging(settings.log_level)

logger = logging.getLogger(__name__)

app = FastAPI(title="Finance Analytics Service")
app.include_router(router)

def domain_status_code(exc: DomainError) -> int:
    if isinstance(exc, InvalidPeriodError):
        return 422

    if isinstance(
        exc,
        (AccountNotFoundError, NoTransactionsFoundError),
    ):
        return 404

    if isinstance(
        exc,
        (PredictionUnavailableError, RepositoryUnavailableError),
    ):
        return 503

    return 400

@app.exception_handler(DomainError)
async def handle_domain_error(
    request: Request,
    exc: DomainError,
```

```

) -> JSONResponse:
    request_id = request.headers.get("X-Request-ID")

    return JSONResponse(
        status_code=domain_status_code(exc),
        content={
            "code": exc.code,
            "message": exc.public_message,
            "request_id": request_id,
        },
    )

@app.exception_handler(Exception)
async def handle_unexpected_error(
    request: Request,
    exc: Exception,
) -> JSONResponse:
    request_id = request.headers.get("X-Request-ID")

    logger.exception(
        "unexpected_request_failure",
        extra={
            "event": "unexpected_request_failure",
            "request_id": request_id,
            "path": request.url.path,
        },
    )

    return JSONResponse(
        status_code=500,
        content={
            "code": "internal_error",
            "message": "The request could not be completed.",
            "request_id": request_id,
        },
    )

```

Production correction

The generated request ID should ideally be installed in middleware so the same value is available to:

- The route.
- Exception handlers.
- Logs.
- Traces.
- Response headers.

The skeleton keeps this simpler for Day 1.

11.15 Unit test

```
# tests/unit/test_analytics_service.py

from datetime import UTC, datetime
from decimal import Decimal

import pytest

from app.clients.mock_model import MockRiskModelClient
from app.domain.models import Transaction, TransactionType
from app.repositories.in_memory import InMemoryTransactionRepository
from app.services.analytics import AnalyticsService

@pytest.mark.asyncio
async def test_analyze_calculates_cashflow_and_calls_model() -> None:
    repository = InMemoryTransactionRepository(
        transactions=[
            Transaction(
                transaction_id="credit-1",
                account_id="account-1",
                occurred_at=datetime(
                    2026, 8, 1, 10, 0, tzinfo=UTC
                ),
                amount=Decimal("1000.00"),
                transaction_type=TransactionType.CREDIT,
            ),
            Transaction(
                transaction_id="debit-1",
                account_id="account-1",
                occurred_at=datetime(
                    2026, 8, 2, 10, 0, tzinfo=UTC
                ),
                amount=Decimal("400.00"),
                transaction_type=TransactionType.DEBIT,
            ),
        ]
    )

    service = AnalyticsService(
        repository=repository,
        model_client=MockRiskModelClient(),
        model_timeout_seconds=1.0,
    )

    result = await service.analyze(
        account_id="account-1",
        start=datetime(2026, 8, 1, tzinfo=UTC),
        end=datetime(2026, 9, 1, tzinfo=UTC),
        request_id="request-1",
    )

    assert result.features.total_inflow == Decimal("1000.00")
```

```
assert result.features.total_outflow == Decimal("400.00")
assert result.features.net_cash_flow == Decimal("600.00")
assert result.features.expense_ratio == pytest.approx(0.4)
assert result.features.transaction_count == 2
assert 0.0 <= result.prediction.score <= 1.0
assert result.prediction.model_version == "mock-risk-v1"
```

12. Correctness conditions

The implementation is correct only when the following conditions remain true.

Financial calculation conditions

1. Money is represented using `Decimal`.
2. Every transaction amount is positive.
3. Transaction type determines inflow or outflow.
4. Net cash flow equals inflow – outflow.
5. No transaction is counted outside the requested half-open interval.
6. No transaction belonging to another account is analyzed.
7. Division by zero is handled when inflow is zero.

Model conditions

1. Model score stays between zero and one.
2. Model version is returned with every prediction.
3. Model timeout is finite.
4. A dependency failure does not return a fabricated successful prediction.
5. The mock is never represented as a validated credit-risk model.

API conditions

1. Invalid requests fail before business processing.
 2. Internal database or model details are not exposed.
 3. A request identifier is returned for operational investigation.
 4. Known failures map to stable error codes.
 5. Unexpected failures return a generic 500 response.
-

13. Production trade-offs

Dataclasses versus Pydantic everywhere

Current choice: dataclasses in the domain and Pydantic at boundaries.

Advantages:

- Lower domain coupling.
- Faster and simpler internal objects.

- Clear transport/domain separation.

Cost:

- Mapping code is required.
- Some validation appears in two places.

Using Pydantic everywhere can be reasonable for a smaller service, but it increases framework influence inside the core.

Repository abstraction

Advantages:

- Enables unit tests.
- Supports Postgres, warehouse, or service-backed implementations.
- Keeps SQL out of the use case.

Cost:

- Adds files and abstractions.
- Can become unnecessary ceremony if only one tiny data path will ever exist.

The interface is justified here because persistence replacement and unit testing are realistic requirements.

Synchronous versus asynchronous model calls

Async is useful when the process waits on network I/O.

It does not make CPU-heavy local inference faster by itself. CPU- or GPU-bound inference may need:

- A dedicated serving process.
 - Batching.
 - Multiprocessing.
 - A model server.
 - A queue.
 - GPU-aware scheduling.
-

Fallback behavior

Possible fallback choices include:

- Return analytics without risk prediction.
- Use a previous approved model.
- Queue the request for later.
- Fail the entire request.

The correct choice depends on the product contract.

For a high-consequence financial decision, silently returning a heuristic fallback as though it were the primary model would be unsafe. The skeleton fails explicitly.

Model call placement

The service directly calls the model client because the workflow is small.

For a larger platform, you might introduce:

- Feature service.
- Policy service.
- Model router.
- Evaluation hook.
- Result store.
- Workflow engine.

Do not introduce these before their operational value is clear.

14. Failure modes and important pitfalls

Data and analytical failures

- Duplicate transactions inflate totals.
- Late-arriving transactions change historical results.
- Currency differences are ignored.
- Refunds and reversals are incorrectly classified.
- Pending and settled transactions are mixed.
- Zero-inflow handling creates misleading risk scores.
- Future information leaks into training features.
- Account closure or inactivity is misinterpreted as low risk.

Model failures

- Training and serving features differ.
- Score calibration degrades.
- Model version is not recorded.
- Thresholds are changed without evaluation.
- Drift is detected but no action is defined.
- The service retries non-idempotent model or tool operations.
- A heuristic fallback is presented as a real model prediction.

Backend failures

- No deadline around the model call.
- Retries multiply load during an outage.

- Connection pools are exhausted.
- Raw account identifiers appear in logs.
- Error handlers reveal stack traces.
- Configuration is read repeatedly inside hot request paths.
- Global mutable state causes test interference.
- Request IDs differ between route, logs, and response.
- Cross-tenant repository filtering is incomplete.

Architecture pitfalls

- Interfaces are created for every function without evidence of variation.
 - Domain logic leaks into FastAPI routes.
 - Vendor-specific model payloads spread through the application.
 - `utils.py` becomes the actual business layer.
 - Framework models are passed directly into database code.
 - Every model failure is treated as a generic 500.
 - The service catches all exceptions and continues with incomplete output.
-

15. Day 1 interview discussion points

“What does backend engineering add to an ML system?”

A strong answer:

“The model supplies a prediction or generation capability, but the backend turns it into a reliable product. It owns validated contracts, feature and data access, workflow orchestration, timeouts, retries, authorization, version tracking, observability, and safe failure behavior. For an applied AI system, I treat model quality and service reliability as separate requirements and test both.”

“Why use repository and model-client interfaces?”

“The use case should depend on the capabilities it needs rather than Postgres or a particular model provider. Narrow interfaces make the orchestration independently testable, support provider replacement, and prevent infrastructure concerns from spreading into the domain. I would avoid the abstraction when there is no realistic variation or testing benefit.”

“Why not put everything in the FastAPI route?”

“The route is a transport adapter. Keeping orchestration in an application service allows the same workflow to be used from HTTP, a worker, or a scheduled job, and it makes domain behavior testable without starting the web framework.”

“How would you productionize the mock model?”

Replace it with an implementation that adds:

- Authenticated model endpoint access.

- Explicit request and response schemas.
- Model and feature versions.
- Timeouts and bounded retries.
- Circuit breaking.
- Prediction validation.
- Metrics and tracing.
- Shadow or canary evaluation.
- Rollback support.
- Drift and calibration monitoring.
- Governance approval appropriate to the financial use case.

16. Initial 63-day baseline scorecard

Scores below are **preparation estimates**, not claims of formal assessment.

Scale:

- 1 — major gap.
- 2 — conceptual or limited evidence.
- 3 — working interview capability.
- 4 — strong senior-level capability.
- 5 — consistently strong under difficult interview follow-ups.

Competency	Day 1 baseline	Day 63 target	Current basis
Role and problem framing	3.0	4.5	Strong system ownership; needs more structured product/ML framing
Analytics and statistics	2.5	4.0	Some analytics and data experience; limited detailed case-study evidence
SQL and data reasoning	3.0	4.0	Database and data-platform experience; interview-depth SQL should be tested
Classical ML	2.5	4.0	Prior ML pipelines and inference; needs rigorous experimental depth
Forecasting	1.0	3.5	No explicit resume evidence
Deep-learning fundamentals	2.5	3.5	Framework and training exposure; theory and architecture depth need revision
RAG and retrieval	4.0	4.7	Strong production evidence
Agent architecture	3.5	4.5	Strong tooling and orchestration; deepen safety and state reasoning
Governance and responsible AI	2.5	4.0	Runtime safeguards exist; formal governance evidence is limited
Python backend engineering	4.0	4.7	Strong APIs, concurrency, testing, and microservices evidence

MLOps and production systems	3.8	4.5	Strong deployment and reliability; deepen lifecycle and incident discussion
Applied-AI system design	3.7	4.6	Strong production architecture; broaden beyond RAG
Coding and DSA	3.0	4.2	Good Python foundation; timed problem-solving needs practice
Leadership and behavioral	3.0	4.3	Leadership and ownership present; stories require stronger detail
Communication	3.3	4.5	Technical breadth exists; answers need structured, concise delivery

Weekly tracking formula

At the end of every seven days, score each competency using four dimensions:

$$\text{Competency score} = 0.25K + 0.25I + 0.25P + 0.25E$$

Where:

- K = knowledge.
- I = interview explanation.
- P = practical implementation.
- E = verified evidence from your experience or portfolio.

This prevents theoretical study from being mistaken for interview readiness.

17. Weak-area register

Weak area	Priority	Risk in interview	63-day evidence required
Forecasting	Critical	Cannot design time-aware validation or forecasting system	One complete forecasting project with naive baseline, backtesting, features, intervals, metrics, and deployment discussion
Classical ML experimentation	Critical	Answers remain library-level rather than senior-level	One tabular case comparing baselines, tree models, validation, thresholding, calibration, explainability, and drift
Analytics/statistical reasoning	High	May jump to models before defining the decision problem	Repeated case drills covering target, population, leakage, metrics, bias, and business action
Coding speed and correctness	High	Strong architecture but incomplete timed implementation	Timed DSA practice plus explicit invariants, complexity, and tests
Governance	High	Guardrails discussed without lifecycle governance	Governance checklist covering data, model, prompt, retrieval, tools, approvals, audits, and incidents

Leadership evidence	High	“Led” claims may sound vague under follow-up	Five verified STAR stories with team context, personal actions, conflict, outcome, and lessons
Non-RAG AI system design	Medium-high	Profile appears narrowly GenAI-focused	Designs for forecasting, ranking, anomaly detection, batch prediction, and online classification
Deep-learning fundamentals	Medium	Framework familiarity without theoretical depth	Clear explanations of optimization, regularization, transformers, fine-tuning, and inference
Google-cloud AI platform mapping	Medium	Architecture not translated into target ecosystem	Map existing concepts to Vertex AI, Gemini, data, deployment, monitoring, and governance components
Resume evidence precision	Medium	Risk of overclaiming metrics or ownership	Evidence sheet marking each claim as verified, conceptual, collaborative, or placeholder

18. Day 1 completion checklist

By the end of today, you should be able to do the following without notes:

1. Explain the eight role competencies in approximately two minutes.
2. State your strongest three areas: production Python/backend, RAG, and agentic system delivery.
3. State your primary gaps honestly: forecasting, rigorous classical ML, formal governance, and timed coding.
4. Explain why model quality alone is insufficient for production.
5. Draw the layered finance-service architecture.
6. Explain dataclasses versus Pydantic.
7. Explain constructor dependency injection.
8. Explain all five SOLID principles using the finance service.
9. Classify validation, domain, dependency, and programming errors.
10. Implement or review the service skeleton and its unit test.
11. Give a 90-second explanation of how your DPDK project demonstrates applied-AI production engineering without adding unverified metrics.

Day 1 Coding Block — Arrays

Add this coding block to Day 1 after the Python backend foundation section.

Topics to refresh

Array traversal

Common traversal patterns:

```
for value in nums:
    ...
```

```

for index, value in enumerate(nums):
    ...

for index in range(len(nums)):
    ...

for index in range(len(nums) - 1, -1, -1):
    ...

```

Choose based on the operation:

- Use direct traversal when only values matter.
- Use `enumerate` when both index and value matter.
- Use an index loop when updating positions or comparing neighboring elements.
- Use reverse traversal for suffix calculations or safe in-place updates.

In-place operations

An in-place algorithm modifies an existing array rather than allocating another array proportional to the input size.

```

def reverse_in_place(nums: list[int]) -> None:
    left, right = 0, len(nums) - 1

    while left < right:
        nums[left], nums[right] = nums[right], nums[left]
        left += 1
        right -= 1

```

Complexity:

- Time: $O(n)$
- Auxiliary space: $O(1)$

Important interview distinction:

The returned output array usually does not count as auxiliary space when the problem requires that output. Any additional array proportional to n does count.

Prefix and suffix ideas

A prefix value summarizes everything before or through an index.

```

nums:           [2, 3, 4, 5]
prefix product: [1, 2, 6, 24]

```

Here, `prefix[i]` represents the product of elements strictly before index i .

A suffix value summarizes everything after or from an index.

```
suffix product:  [60, 20, 5, 1]
```

Prefix/suffix techniques are recognition signals for problems asking about:

- Everything except the current element.
- Range sums or products.
- Left-side and right-side information.
- Equilibrium or pivot indexes.
- Trapping rainwater.
- Maximum contribution from both directions.
- Repeated range queries.

Medium problem — Product of Array Except Self

Problem

Given an integer array `nums`, return an array `answer` where:

$$\text{answer}[i] = \prod_{j \neq i} \text{nums}[j]$$

Do not use division.

The optimized solution must run in $O(n)$ time.

Sample input and output

```
Input:  [1, 2, 3, 4]
Output: [24, 12, 8, 6]
```

1. Recognition signals

The important phrases are:

- “Product of every element except itself.”
- “Do not use division.”
- “Linear-time solution.”
- Each answer requires information from both the left and right sides.

For index i , the answer can be decomposed as:

$$\text{answer}[i] = (\text{product of elements before } i) \times (\text{product of elements after } i)$$

That directly suggests prefix and suffix products.

2. Brute-force reasoning

For every index:

1. Traverse the entire array.
2. Multiply every element except the current one.
3. Store the product.

Pseudocode

```
CREATE answer array

FOR each index i:
    product = 1

    FOR each index j:
        IF i is not equal to j:
            product = product * nums[j]

    APPEND product to answer

RETURN answer
```

Brute-force complexity

- Time: $O(n^2)$
- Auxiliary space: $O(1)$, excluding the output array

Why it is insufficient

The same elements are multiplied repeatedly. For an array of size n , approximately (n^2) operations are performed.

3. Intermediate prefix/suffix solution

Create two arrays:

- `prefix[i]`: product of all elements before i
- `suffix[i]`: product of all elements after i

Then:

```
answer[i] = prefix[i] * suffix[i]
```

Prefix/suffix complexity

- Time: $O(n)$
- Auxiliary space: $O(n)$

This is correct, but the two additional arrays can be avoided.

4. Optimized reasoning

Use the required output array to store prefix products.

Left-to-right pass

For:

```
nums = [1, 2, 3, 4]
```

Build:

```
answer = [1, 1, 2, 6]
```

Meaning:

- Before index 0: product is 1
- Before index 1: 1
- Before index 2: $1 \times 2 = 2$
- Before index 3: $1 \times 2 \times 3 = 6$

Right-to-left pass

Maintain one variable called `suffix_product`.

Initially:

```
suffix_product = 1
```

At each position:

1. Multiply `answer[i]` by the current suffix product.
2. Include `nums[i]` in the suffix product for the next position to the left.

```
Initial answer:      [1, 1, 2, 6]
```

```
Process index 3:  
answer[3] = 6 × 1  
suffix = 1 × 4
```

```
Process index 2:  
answer[2] = 2 × 4  
suffix = 4 × 3
```

```
Process index 1:  
answer[1] = 1 × 12  
suffix = 12 × 2
```

```
Process index 0:  
answer[0] = 1 × 24
```

Final result:

```
[24, 12, 8, 6]
```

5. Optimized pseudocode

```
FUNCTION product_except_self(nums):  
    n = length of nums  
    answer = array of size n filled with 1  
  
    prefix_product = 1  
  
    FOR index from 0 to n - 1:  
        answer[index] = prefix_product  
        prefix_product = prefix_product * nums[index]  
  
    suffix_product = 1  
  
    FOR index from n - 1 down to 0:  
        answer[index] = answer[index] * suffix_product  
        suffix_product = suffix_product * nums[index]  
  
    RETURN answer
```

6. Edge cases

One zero

```
Input:  [1, 2, 0, 4]  
Output: [0, 0, 8, 0]
```

Only the position containing zero receives the product of all non-zero values.

Two zeros

```
Input:  [1, 0, 3, 0]  
Output: [0, 0, 0, 0]
```

Every result includes at least one zero.

Negative values

```
Input:  [-1, 2, -3, 4]  
Output: [-24, 12, -8, 6]
```

The algorithm naturally handles signs.

Two elements

```
Input:  [5, 7]  
Output: [7, 5]
```

Ones

```
Input:  [1, 1, 1]
Output: [1, 1, 1]
```

Input contract

The standard problem normally guarantees at least two elements. In production code, decide whether an empty or one-element input should:

- Raise a validation error.
- Return an empty result.
- Follow a documented mathematical convention.

The implementation below enforces at least two elements.

Python solution

```
from collections.abc import Sequence

def product_except_self(nums: Sequence[int]) -> list[int]:
    """
    Return the product of all elements except the element at each index.

    Time complexity: O(n)
    Auxiliary space: O(1), excluding the returned output list
    """
    if len(nums) < 2:
        raise ValueError("nums must contain at least two elements")

    answer = [1] * len(nums)

    prefix_product = 1
    for index, value in enumerate(nums):
        answer[index] = prefix_product
        prefix_product *= value

    suffix_product = 1
    for index in range(len(nums) - 1, -1, -1):
        answer[index] *= suffix_product
        suffix_product *= nums[index]

    return answer
```

Key design decisions

Why initialize with 1?

One is the multiplicative identity: $x \times 1 = x$.

At the first index, no elements exist on the left, so the left-side product is 1. The same applies to the right side of the final index.

Why update the suffix after the answer?

At index i , `suffix_product` must contain only elements strictly after i .

Therefore, the order must be:

```
answer[index] *= suffix_product
suffix_product *= nums[index]
```

Reversing these lines would incorrectly include the current value.

Why not modify `nums`?

The original input is needed during both passes. Modifying it would destroy information required for later calculations.

The output array is reused as prefix storage, which eliminates the need for a separate prefix array.

Correctness argument

During the first pass:

$$\text{answer}[i] = \prod_{j=0}^{i-1} \text{nums}[j]$$

During the reverse pass, before processing index i :

$$\text{suffix_product} = \prod_{j=i+1}^{n-1} \text{nums}[j]$$

Multiplying them gives:

$$\text{answer}[i] = \left(\prod_{j=0}^{i-1} \text{nums}[j] \right) \left(\prod_{j=i+1}^{n-1} \text{nums}[j] \right)$$

This is exactly the product of every element except `nums[i]`.

Python solution complexity

- Time: $O(n)$
 - One forward traversal.
 - One reverse traversal.
 - Auxiliary space: $O(1)$, excluding the output list.
 - Required output space: $O(n)$.
-

Python test cases

```
def test_product_except_self() -> None:
    assert product_except_self([1, 2, 3, 4]) == [24, 12, 8, 6]
    assert product_except_self([1, 2, 0, 4]) == [0, 0, 8, 0]
    assert product_except_self([1, 0, 3, 0]) == [0, 0, 0, 0]
    assert product_except_self([-1, 2, -3, 4]) == [-24, 12, -8, 6]
    assert product_except_self([5, 7]) == [7, 5]

    try:
        product_except_self([5])
    except ValueError:
        pass
    else:
        raise AssertionError("Expected ValueError for fewer than two
elements")
```

Go solution review

Use this only after independently solving and explaining the Python version.

```
package arrays

import "fmt"

func ProductExceptSelf(nums []int) ([]int, error) {
    if len(nums) < 2 {
        return nil, fmt.Errorf("nums must contain at least two elements")
    }

    answer := make([]int, len(nums))

    prefixProduct := 1
    for i, value := range nums {
        answer[i] = prefixProduct
        prefixProduct *= value
    }

    suffixProduct := 1
    for i := len(nums) - 1; i >= 0; i-- {
        answer[i] *= suffixProduct
        suffixProduct *= nums[i]
    }

    return answer, nil
}
```

Useful Python-to-Go comparison

Python	Go
<code>answer = [1] * len(nums)</code>	<code>make([]int, len(nums))</code> initializes values to zero
Dynamic integer size	<code>int</code> has a fixed platform-dependent size
Exceptions for invalid input	Explicit <code>error</code> return
<code>enumerate(nums)</code>	<code>for i, value := range nums</code>
Negative-step <code>range</code>	Traditional reverse index loop

The Go output slice starts with zeros, but every element is assigned a prefix product before it is used.

Backend-related Go consideration

With large values, integer multiplication may overflow silently in Go. Production alternatives include:

- Validate bounds before multiplication.
- Use `int64` when its range is sufficient.
- Use `math/big.Int` for arbitrary precision.
- Define overflow behavior in the API contract.

Python integers grow automatically, although extremely large values still increase memory and CPU cost.

Independent-solve checklist

Before reviewing either solution, your interview response should cover:

1. State the $O(n^2)$ brute-force approach.
2. Recognize that each result combines left and right information.
3. Propose prefix and suffix arrays.
4. Optimize by storing prefixes in the output array.
5. Maintain a single suffix variable.
6. Explain why the update order matters.
7. Test zero, multiple-zero, negative, and two-element cases.
8. State $O(n)$ time and $O(1)$ auxiliary space.
9. Give a brief correctness argument using prefix and suffix invariants.